

Lab 10

Stephanie Salgado

Setup

Downloading OMNeT++



Since I already had OMNeT++ installed for the term project, I don't have to download or install it again, but I will show how it can be done. It can be downloaded from <https://omnetpp.org/download/>.

Installing OMNeT++

```
stephanie@ubuntu:~$ cd ~/Downloads
stephanie@ubuntu:~/Downloads$ ls
omnetpp-6.1.0-linux-x86_64.tgz
stephanie@ubuntu:~/Downloads$ tar xvfz omnetpp-6.1.0-linux-x86_64.tgz
```

Next, you can navigate to "Downloads" and unarchive it. After this, you would have to install components needed for OMNeT++.

```
stephanie@ubuntu:~/Downloads$ sudo apt-get install build-essential clang lld gdb  
bison flex perl python3 python3-pip qtbase5-dev qtchooser qt5-qmake qtbase5-dev  
-tools libqt5opengl5-dev libxml2-dev zlib1g-dev doxygen graphviz libwebkit2gtk-4  
.0-37 xdg-utils
```

```
stephanie@ubuntu:~/Downloads$ python3 -m pip install --user --upgrade numpy pand  
as matplotlib scipy seaborn posix_ipc
```

These were the ones I needed.

```
stephanie@ubuntu:~/Downloads$ cd omnetpp-6.1/
```

```
stephanie@ubuntu:~/Downloads/omnetpp-6.1$ source setenv
```

```
stephanie@ubuntu:~/Downloads/omnetpp-6.1$ gedit ~/.bashrc
```

After that, you must set the environment variables. Notice these are the commands I used when I installed OMNeT++. The latest version may be different.

```
[ -f "$HOME/omnetpp-6.1/setenv" ] && source "$HOME/omnetpp-6.1/setenv"
```

Once in the "bashrc" file, you can add the following line.

Building OMNeT++

```
stephanie@ubuntu:~/Downloads/omnetpp-6.1$ ./configure
```

```
stephanie@ubuntu:~/omnetpp-6.1$ make
```

From within the "omnetpp-..." directory, use the following commands. Just keep in mind that if any components are missing, the first command will give you errors and won't create the make file. Once everything is properly installed, the "make" command will take a long time to finish.

Launching the OMNeT++ Simulation IDE

```
stephanie@ubuntu:~$ omnetpp
```

If everything was done correctly, "omnetpp" should start the OMNeT++ IDE.



The first launch will ask if you want to install INET, I did. If this step was skipped, you can install the framework through OMNeT++ in the workbench by going to “Help” > “Install Simulation Models”. If not, it can be installed independently.

Tictoc

```
stephanie@ubuntu:~$ cd ~/omnetpp-6.1/samples/tictoc
stephanie@ubuntu:~/omnetpp-6.1/samples/tictoc$ ./tictoc
OMNeT++ Discrete Event Simulation (C) 1992-2024 Andras Varga, OpenSim Ltd.
Version: 6.1, build: 241008-f7568267cd, edition: Academic Public License -- NOT
FOR COMMERCIAL USE
See the license for distribution terms and warranty disclaimer

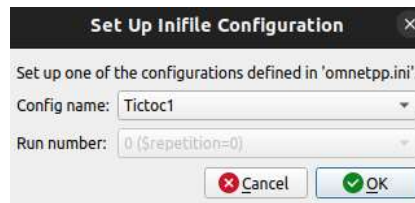
Setting up Qtenv...

Loading NED files from .: 19

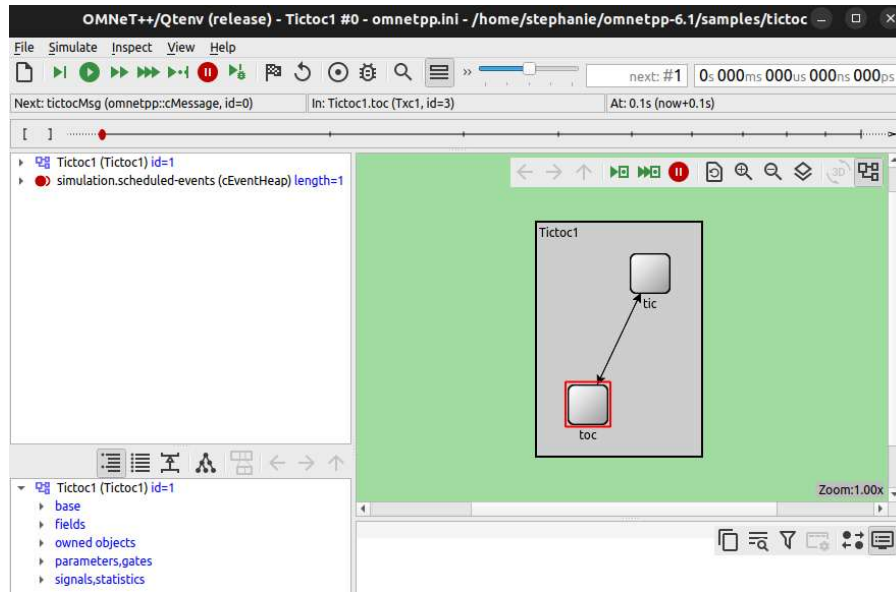
Loading images from 'images': *: 0
Loading images from '/home/stephanie/omnetpp-6.1/images': *: 0 abstract/*: 90
background/*: 4 block/*: 325 device/*: 195 logo/*: 3 maps/*: 9 misc/*: 70
msg/*: 55 old/*: 111 status/*: 28

Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland
to run on Wayland anyway.
```

To begin this demo, I navigate to the “tictoc” sample provided with the installation of OMNeT++. Then, I use “./tictoc” to run it.



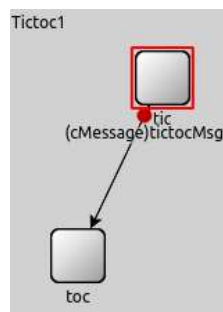
After that, there is a prompt to set up a configuration. I go with the default.



This is what it looks like afterwards. Notice there are 2 nodes, "tic" and "toc".



The simulation can be run by hitting the green "play" button.



In this screenshot, we can observe a packet being sent from the "tic" to "toc".

```

** Event #1 t=0.1 Tictoc1.tic (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #2 t=0.2 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #3 t=0.3 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #4 t=0.4 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #5 t=0.5 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #6 t=0.6 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #7 t=0.7 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #8 t=0.8 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #9 t=0.9 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #10 t=1 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #11 t=1.1 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #12 t=1.2 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)
** Event #13 t=1.3 Tictoc1.toc (Txcl, id=3) on tictocMsg (omnetpp::cMessage, id=0)
** Event #14 t=1.4 Tictoc1.tic (Txcl, id=2) on tictocMsg (omnetpp::cMessage, id=0)

```

Each event is displayed below as the simulation goes on.

Event#	Time	Relevant Hops	Name
#0	0.000	tic → toc	tictocMsg
#1	0.100	toc → tic	tictocMsg
#2	0.200	tic → toc	tictocMsg
#3	0.300	toc → tic	tictocMsg
#4	0.400	tic → toc	tictocMsg
#5	0.500	toc → tic	tictocMsg
#6	0.600	tic → toc	tictocMsg
#7	0.700	toc → tic	tictocMsg
#8	0.800	tic → toc	tictocMsg
#9	0.900	toc → tic	tictocMsg
#10	1.000	tic → toc	tictocMsg
#11	1.100	toc → tic	tictocMsg
#12	1.200	tic → toc	tictocMsg

The view of these events can be changed. In this view, the events are displayed in a more intuitive way. Here we can tell “tic” sends a packet to “toc” and “toc” sends a reply. This behavior continues on throughout the simulation.



Clicking the “checkered flag” icon concludes the simulation.

```

stephanie@ubuntu:~/omnetpp-6.1/samples/tictoc/results$ ls
Tictoc1-#0.sca

```

Notice the results are automatically saved in “.../samples/tictoc/results”.

```

simple Txcl
{
    gates:
        input in;
        output out;
}

// Two instances (tic and toc) of Txcl connected both ways.
// Tic and toc will pass messages to one another.
network Tictoc1
{
    submodules:
        tic: Txcl;
        toc: Txcl;
    connections:
        tic.out --> { delay = 100ms; } --> toc.in;
        tic.in <-- { delay = 100ms; } <-- toc.out;
}

```

Here, we can see that the “tictoc1.ned” file defines the simple network model. It includes two modules, “tic” and “toc,” both based on the Txcl module. These modules are connected so that they send messages to each other with a 100ms delay, which we observed.


```

*/
class Txc1 : public cSimpleModule
{
protected:
    // The following redefined virtual function holds the algorithm.
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
};

// The module class needs to be registered with OMNeT++
Define_Module(Txc1);

void Txc1::initialize()
{
    // Initialize is called at the beginning of the simulation.
    // To bootstrap the tic-toc-tic-toc process, one of the modules needs
    // to send the first message. Let this be 'tic'.

    // Am I Tic or Toc?
    if (strcmp("tic", getName()) == 0) {
        // create and send first message on gate "out". "tictocMsg" is an
        // arbitrary string which will be the name of the message object.
        cMessage *msg = new cMessage("tictocMsg");
        send(msg, "out");
    }
}

void Txc1::handleMessage(cMessage *msg)
{
    // The handleMessage() method is called whenever a message arrives
    // at the module. Here, we just send it to the other module, through
    // gate 'out'. Because both 'tic' and 'toc' does the same, the message
    // will bounce between the two.
    send(msg, "out"); // send out the message
}

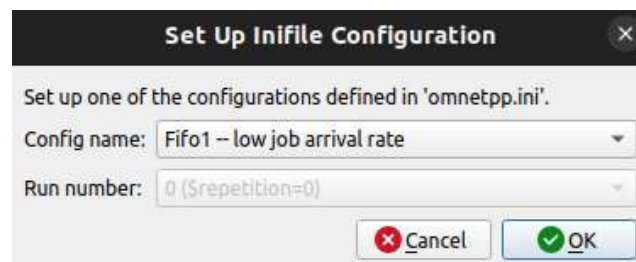
```

Looking closer at the “txc1.cc” file, we can see that it defines the behavior of the Txc1 module, which is used in the “tictoc1.ned” network. The module is derived from “cSimpleModule” and contains functions “initialize()” and “handleMessage(cMessage *msg)”. In the “initialize()” function, the “tic” module sends the first message to start the communication process. The “handleMessage()” function is called whenever a message is received, and it simply sends the message to the other module, creating a loop where messages bounce back and forth between the “tic” and “toc” modules.

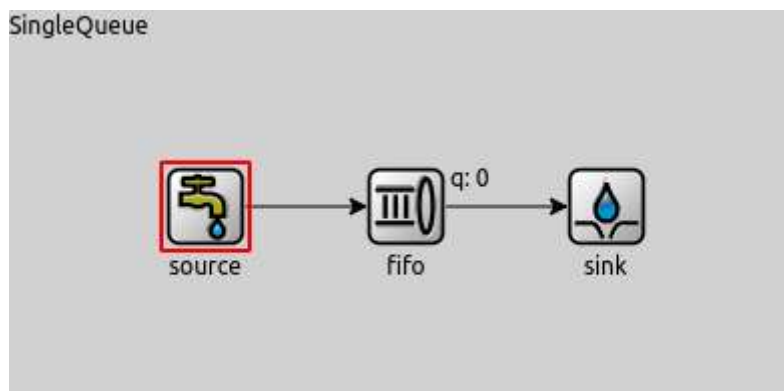
FIFO

```
stephanie@ubuntu:~/omnetpp-6.1/samples/tictoc$ cd ~/omnetpp-6.1/samples/fifo
stephanie@ubuntu:~/omnetpp-6.1/samples/fifo$ ./fifo
```

Similarly, to begin this demo, I navigate to the “FIFO” sample provided with the installation of OMNeT++. Then, I use “./fifo” to run it.



Once again, I go with the default configuration (Fifo1 –low job arrival rate).



Here, we can see the network looks different from the “tictoc” sample. In this simulation, there are three modules, “source”, “queue” and “sink”.



Once again, I run the simulation using the green “play” button. Notice the “job” travels from the “source” through “fifo” and finally to “sink”. It travels in one way.

Event#	Time	Relevant Hops	Name
#177	10.450'900'975'530	source → fifo	job
#179	10.550'900'975'530	fifo → sink	job
#181	10.574'686'867'455	source → fifo	job
#183	10.674'686'867'455	fifo → sink	job
#185	10.721'679'751'693	source → fifo	job
#187	10.821'679'751'693	fifo → sink	job
#189	11.024'986'946'491	source → fifo	job
#191	11.124'986'946'491	fifo → sink	job
#193	11.252'174'734'833	source → fifo	job
#195	11.277'349'616'721	source → fifo	job
#197	11.352'174'734'833	fifo → sink	job
#199	11.452'174'734'833	fifo → sink	job
#201	11.532'395'157'166	source → fifo	job
#203	11.632'395'157'166	fifo → sink	job

This traffic is reflected when viewing the events. I did find it interesting that the even event numbers were skipped. I allowed the simulation to run for a while longer before concluding it with the “checkered flag” button.

```
stephanie@ubuntu:~/omnetpp-6.1/samples/fifo/results$ ls
Fifo1-#0.sca  Fifo1-#0.vci  Fifo1-#0.vec
```

The results are again automatically saved in the respective “results” directory.

```
network SingleQueue
{
    submodules:
        source: Source {
            @display("p=100,100");
        }
        fifo: Fifo {
            @display("p=200,100");
        }
        sink: Sink {
            @display("p=300,100");
        }
    connections:
        source.out --> fifo.in;
        fifo.out --> sink.in;
}
```

The “SingleQueue.ned” file defines a simple queueing network consisting of three modules: “source”, “fifo”, and “sink”. The “source” module generates traffic, which is sent to the “fifo” module, a queueing system. After processing, the traffic is passed to the “sink” module. The modules are connected in sequence: “source” to “fifo”, and “fifo” to “sink”. Each module is displayed at specific positions in the network.


```

simple Source
{
    parameters:
        volatile double interarrivalTime @unit(s);
        @display("i=block/source");
    gates:
        output out;
}

```

The "Source.ned" file defines a simple module called "Source", which generates jobs (messages) at a specified interarrival time. The interarrival time can be a random value, and the module has an output gate to send the generated messages. The display of the module is represented by a source icon ("i=block/source").

```

class Source : public cSimpleModule
{
private:
    cMessage *sendMessageEvent = nullptr;

public:
    virtual ~Source();

protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
};

Define_Module(Source);

Source::~Source()
{
    cancelAndDelete(sendMessageEvent);
}

void Source::initialize()
{
    sendMessageEvent = new cMessage("sendMessageEvent");
    scheduleAt(simTime(), sendMessageEvent);
}

void Source::handleMessage(cMessage *msg)
{
    ASSERT(msg == sendMessageEvent);

    cMessage *job = new cMessage("job");
    send(job, "out");

    scheduleAt(simTime()+par("interarrivalTime").doubleValue(), sendMessageEvent);
}

```

Looking at "Source.cc", the file defines the behavior of the "Source" module. It generates messages (or jobs) at a specified interarrival time. In the "initialize()" function, it schedules the first message to be sent at the start of the simulation. The "handleMessage()" function creates a new message, sends it out through the "output" gate, and then schedules the next message to be sent after the defined interarrival time. The module uses a "sendMessageEvent" to control the timing of message generation.

```

simple Fifo
{
    parameters:
        volatile double serviceTime @unit(s);
        @display("i=block/activeq;q=queue");
        @signal[qlen](type="long");
        @signal[busy](type="bool");
        @signal[queueingTime](type="simtime_t");
        @statistic[qlen](title="queue
length";record=vector,timeavg,max;interpolationmode=sample-hold);
        @statistic[busy](title="server busy
state";record=vector?,timeavg;interpolationmode=sample-hold);
        @statistic[queueingTime](title="queueing
time";unit=s;record=vector,mean,max;interpolationmode=none);
    gates:
        input in;
        output out;
}

```

Here's an explanation of "Fifo.ned". The file defines a simple module called "Fifo", which models a queue with a service time and tracks several parameters. It has "input" and "output" gates, and signals to measure the queue length, server status, and queueing time. The module also records statistics for these signals, such as the average queue length, server activity, and the time spent in the queue.

```

namespace fifo {
    Define_Module(Fifo);

    simtime_t Fifo::startService(cMessage *msg)
    {
        EV << "Starting service of " << msg->getName() << endl;
        return par("serviceTime");
    }

    void Fifo::endService(cMessage *msg)
    {
        EV << "Completed service of " << msg->getName() << endl;
        send(msg, "out");
    }
}; //namespace

```

The "Fifo.cc" file defines how the "Fifo" module works. It includes the "startService()" function, which starts the service for a message and returns the service time. The "endService()" function completes the service and sends the message to the output. Both functions are part of the "fifo" namespace.

```

simple Sink
{
    parameters:
        @display("i=block/sink");
        @signal[lifetime](type="simtime_t");
        @statistic[lifetime](title="lifetime of arrived jobs"; unit=s; record=vector,mean,max;
interpolationmode=none);
    gates:
        input in;
}

```

Next, the "Sink.ned" file defines a simple module called "Sink", which receives and discards jobs (messages) while collecting statistics on their lifetime. It has an input gate to receive messages and tracks the lifetime of each message. The module also records statistics for the lifetime of the messages, including the mean and maximum values. The display of the module is represented by a sink icon ("i=block/sink").

```

#include <omnetpp.h>
using namespace omnetpp;

namespace fifo {
    /**
     * Packet sink; see NED file for more info.
     */
    class Sink : public cSimpleModule
    {
    private:
        simsignal_t lifetimeSignal;

    protected:
        virtual void initialize() override;
        virtual void handleMessage(cMessage *msg) override;
    };

    Define_Module(Sink);

    void Sink::initialize()
    {
        lifetimeSignal = registerSignal("lifetime");
    }

    void Sink::handleMessage(cMessage *msg)
    {
        simtime_t lifetime = simTime() - msg->getCreationTime();
        EV << "Received " << msg->getName() << ", lifetime: " << lifetime << "s" << endl;
        emit(lifetimeSignal, lifetime);
        delete msg;
    }
}; //namespace

```

Finally, "Sink.cc" defines the behavior of the "Sink" module. It calculates the lifetime of each received message by subtracting its creation time from the current simulation time. In the "initialize()" function, it registers a signal to track the lifetime of the messages. The "handleMessage()" function processes each received message, calculates its lifetime, emits the lifetime signal, and then deletes the message. The module collects and reports statistics on the message lifetime.

Summary

In this lab, I used OMNeT++ to run two sample simulations, "Tictoc" and "FIFO". In the "Tictoc" simulation, two modules, "tic" and "toc", send messages back and forth with a 100ms delay. I looked at files like "tictoc1.ned" and "txc1.cc" to understand how this works. Then, I ran the "FIFO" simulation, where messages are created by the "source" module, go through the "fifo" queue, and are discarded by the "sink" module. I looked at the "Source.ned", "Fifo.ned", "Sink.ned" and related ".cc" files to see how the messages are managed. This lab helped me understand how OMNeT++ simulations work and how messages move between modules. Even though I didn't look at the ".h" files, the other files gave me enough information. Overall, this lab was a good introduction to OMNeT++ for those who have never used it.